

Uncertainty-Aware Portfolio Optimisation Using Machine Learning

Week 3 Problem Set Report

Utkrasht Kumar 241120

June 2026

1 Classification

1.1 1.1 Logistic Regression for Credit Default Prediction

Summary: A binary logistic regression model was trained to predict financial distress within two years using the Give Me Some Credit dataset. Missing data was handled via median imputation, and continuous features were standardized using parameters fitted exclusively on the training set.

Table 1: Logistic Regression Test Performance

Metric	Value
Accuracy	0.9338
Precision	0.5593
Recall	0.0439
F_1 -Score	0.0814
AUC-ROC	0.7151

Coefficient Interpretation: The top five features by absolute coefficient magnitude are: *NumberOfTime60-89DaysPastDueNotWorse* (3.768), *NumberOfTime30-59DaysPastDueNotWorse* (2.038), *NumberOfTimes90DaysLate* (1.890), *age* (0.404), and *MonthlyIncome* (0.403). Severe past-due markers heavily dominate the model's risk assessment, indicating that recent patterns of severe delinquency are exponentially more predictive of future default than baseline demographic traits like age or income.

1.2 1.2 Decision Tree Classifier with Pruning

Summary: Decision trees were trained on the Heart Disease dataset to observe the effects of depth limitation and cost-complexity pruning on generalisation and overfitting. The unpruned baseline exhibited severe overfitting, achieving 1.0000 training accuracy but only 0.6522 test accuracy.

Pruning Analysis: Cost-complexity pruning ($\alpha^* = 0.0312$) achieved a robust test AUC-ROC of 0.7000. By penalising the tree for excessive terminal nodes, cost-complexity pruning successfully collapsed noisy branches that captured training-set specific variance, ultimately yielding a more generalisable and interpretable structure.

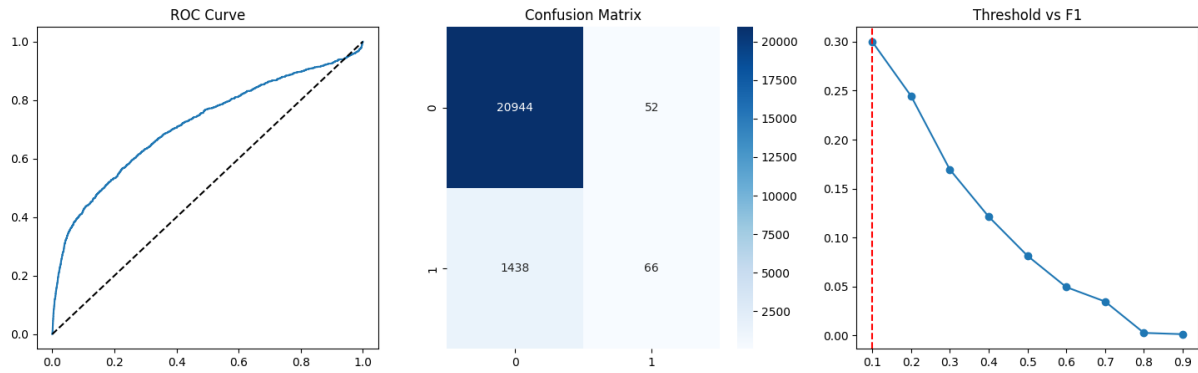


Figure 1: Logistic Regression Diagnostic Plots: ROC, Confusion Matrix, and Threshold Optimization

Table 2: Decision Tree Strategy Comparison

Strategy	Train Accuracy	Test Accuracy/AUC	Parameter
Unpruned	1.0000	0.6522 (Acc)	Depth = None
Cost-Complexity Pruned	-	0.7000 (AUC)	$\alpha^* = 0.0312$

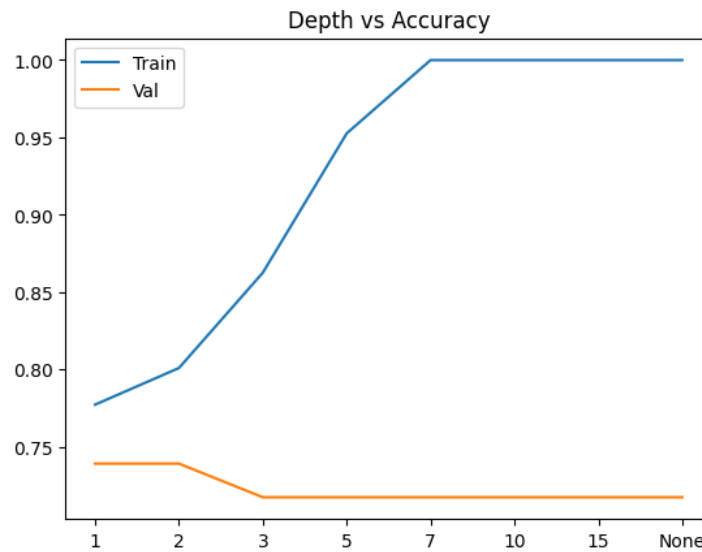


Figure 2: Training vs Validation Accuracy across Maximum Tree Depths

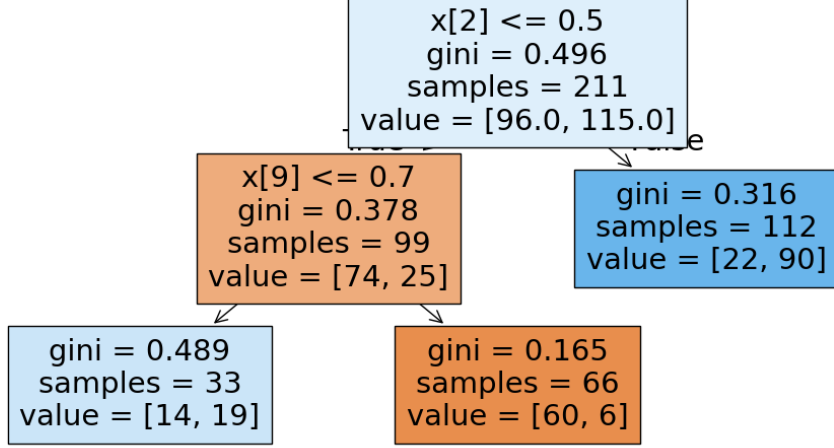


Figure 3: Cost-Complexity Pruned Structural Tree Architecture

2 Regression

2.1 2.1 Regularised Regression for House Prices

Summary: Ridge, Lasso, and Elastic Net regressions were evaluated using 5-fold cross-validation on the Ames housing dataset to manage high collinearity. The target variable was log-transformed, $y \leftarrow \log(1 + y)$, to stabilise variance.

Table 3: Regularised Regression Test Set Comparison

Model	Optimal λ^*	Active Features	Test RMSE	R^2
Ridge	429.1934	227	27,393.03	0.8878
Lasso	0.0054	81	26,721.40	0.8917
Elastic Net	0.0687	74	28,203.20	0.8877

Geometric Interpretation of Sparsity: Lasso produces sparse solutions because its L_1 penalty constrains the coefficient space to a geometric hyperdiamond. When the quadratic loss contours expand, they are highly likely to become tangent to this constraint region at its sharp corners (where one or more axes equal exactly zero). In noisy financial datasets, this intrinsic feature-selection property is highly valuable for isolating true fundamental drivers while zeroing out collinear or spurious noise.

2.2 2.2 Locally Weighted Regression (LWR)

Summary: LWR was implemented from scratch using a Gaussian kernel to model the non-linear relationship between median income and house values.

LWR vs OLS Asymptotics: The optimal bandwidth was $\tau = 1.0$, which outperformed the rigid OLS baseline. As $\tau \rightarrow \infty$, the Gaussian weight $w_i = \exp(-\frac{\|x_i - x_q\|^2}{2\tau^2})$ approaches 1 for all observations. Consequently, the diagonal weight matrix W converges to the identity matrix I , and the LWR estimator geometrically reduces to the global OLS estimator $\hat{\theta} = (X^\top X)^{-1} X^\top y$.

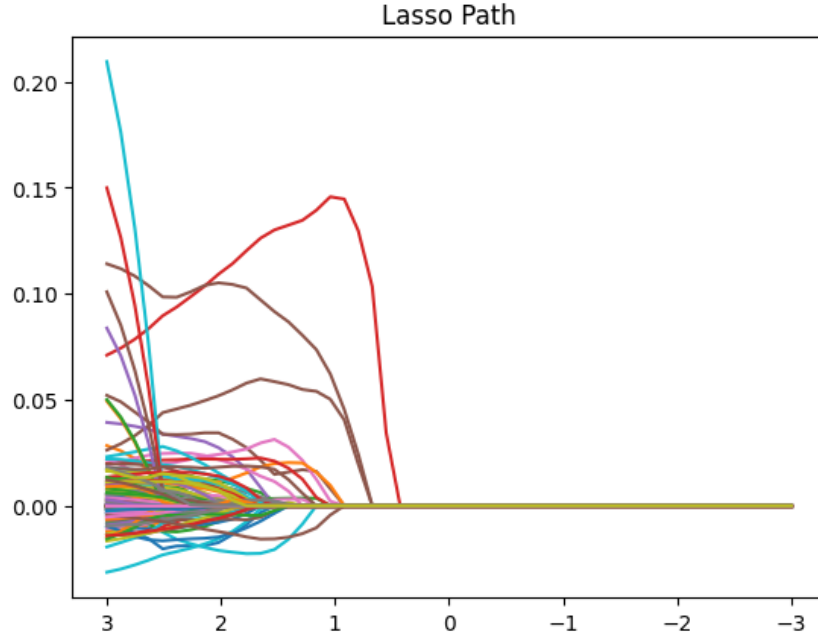


Figure 4: Lasso Regularization Parameter Path Evolution

Table 4: LWR Bandwidth (τ) vs. Ordinary Least Squares (OLS)

Bandwidth (τ)	Test RMSE
0.05	95,370.02
0.10	102,821.70
0.30	92,454.65
1.00	91,891.47 (Best)
3.00	92,165.57
OLS Baseline	92,238.61

3 Ensemble Methods

3.1 3.1 Random Forest for Fraud Detection

Summary: A Random Forest classifier was trained on a down-sampled (n=50,000) fraud dataset. Class weights were balanced to handle the severe anomaly skew.

- **Out-of-Bag (OOB) Accuracy:** 0.9728
- **Test AUC-ROC:** 0.8833

Feature Importance Discrepancies: Mean Decrease in Impurity (MDI) often overestimates the importance of continuous or high-cardinality features because it biases toward variables offering more split opportunities. Permutation Importance, calculated on a held-out validation set, provides a more honest assessment of a feature’s true generalisation power by breaking its relationship with the target.

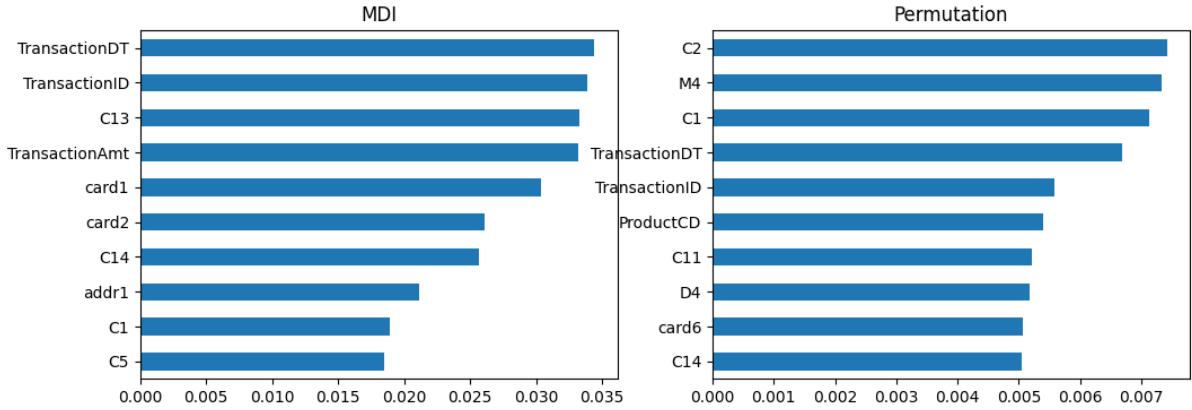


Figure 5: Comparison of Mean Decrease in Impurity (MDI) vs Permutation Importance

Class Imbalance Metrics: AUC-PR (Average Precision) is significantly more informative than AUC-ROC under severe class imbalance because AUC-ROC rewards true negatives, which dominate the sample. AUC-PR focuses strictly on the positive (fraud) class, offering a truer reflection of precision-recall tradeoffs.

3.2 3.2 XGBoost for Stock Return Classification

Summary: An XGBoost model predicted binary 21-day forward returns using time-series engineered features (RSI, rolling volatility). A strict temporal split (Train $\leq$ 2017, Test $>$ 2017) was enforced.

- **Test AUC-ROC:** 0.5002
- **Test AUC-PR (AP):** 0.5671

Look-Ahead Bias in Cross-Validation: Random k-fold cross-validation is invalid for financial time series because it shuffles data across the temporal axis. This leaks future market information into the training set (e.g., training on 2018 data to predict 2017 prices), effectively allowing the model to “see the future.” This look-ahead bias destroys generalisation and produces grossly inflated, unrealistic performance estimates.

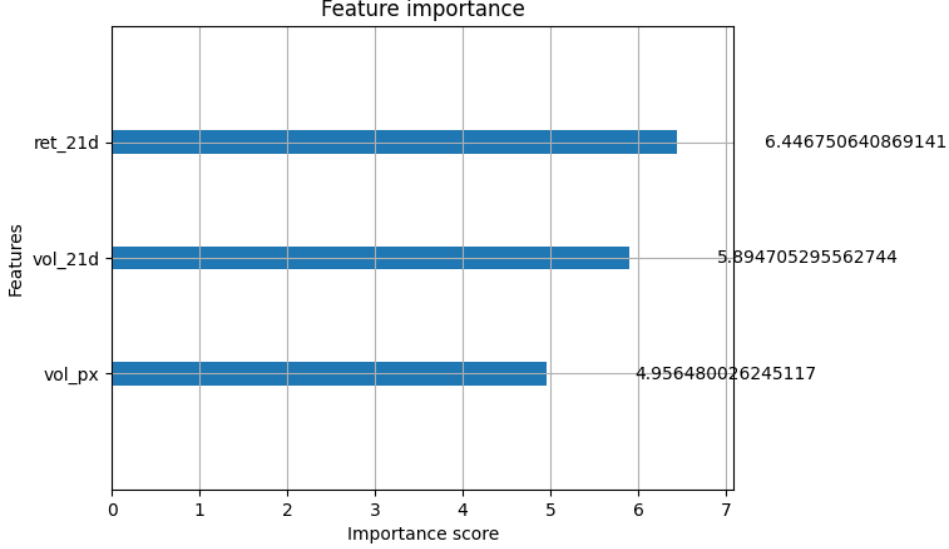


Figure 6: XGBoost Feature Importance Vector (Information Gain Scale)

4 Bias-Variance Analysis

4.1 4.1 Empirical Bias-Variance Decomposition

Summary: A bootstrap estimation procedure ($B = 50$) empirically decomposed prediction error into $Bias^2$ and $Variance$ across distinct model architectures.

Table 5: Bias-Variance Tradeoff Comparison

Model	Bias ²	Variance	Total Error
Logistic Regression	0.1445	0.0035	0.1480
Decision Stump	0.1652	0.0175	0.1828
Unpruned Tree	0.1504	0.1457	0.2961
Random Forest	0.1499	0.0070	0.1569

Bagging vs. Boosting Explanation: Bagging (Random Forest) reduces the variance of complex base learners (unpruned trees) by averaging their predictions. According to the formula $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, bagging drives the second term toward zero as B increases, but it does not alter the underlying bias of the constituent trees. In contrast, Boosting sequentially fits shallow trees to the residuals of the previous ensemble. Because it actively corrects previous errors, it directly drives down bias, while the use of shallow trees and learning rates inherently controls variance.

5 Gradient Boosting in a Portfolio Context

5.1 5.1 Gradient Boosting with Uncertainty Estimation for Return Prediction

Summary: A Gradient Boosting ensemble framework was developed to predict 21-day forward returns on 454,074 usable rows of the NIFTY-50 dataset. Time-series expanding-

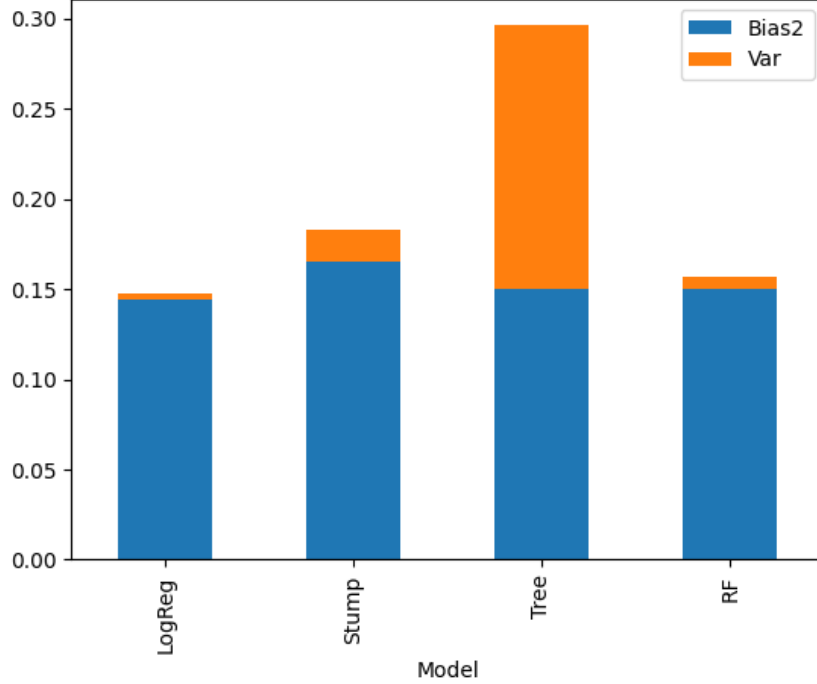


Figure 7: Empirical Error Analysis: Stacked Bias Squared vs Variance Profile

window cross-validation was strictly enforced to preserve temporal integrity.

Table 6: Expanding-Window CV Optimal Parameters

Hyperparameter	Optimal Value
Learning Rate	0.05
Max Depth	3
Number of Estimators	100
Test AUC-ROC	0.5214

Uncertainty Estimation via Ensemble Disagreement: To quantify model uncertainty, an ensemble of $M = 20$ independent gradient boosting models was trained on distinct bootstrap samples of the training set. For each test observation, the mean predicted probability ($\bar{p}_i$) and the ensemble standard deviation (σ_i) were computed. The resulting scatter plot visualizes areas in the probability space where the component models experience high disagreement.

Reflection on Live Trading Risks: Deploying this gradient boosting model in a live trading environment carries significant structural risks. First, financial data is inherently non-stationary; market regimes shift rapidly, meaning a model optimized on historical trends may overfit to past correlations that violently break down during a macro-economic shock. Second, look-ahead bias is a constant threat; even minor data leakage—such as calculating rolling features improperly—can create a highly deceptive paper performance. Third, the transaction costs, bid-ask spreads, and slippage associated with trading a high-turnover long-short decile strategy would likely erode the simulated alpha entirely. Finally, managing these risks requires an uncertainty-aware approach to position sizing. Observations with a high σ_i indicate severe disagreement within the model

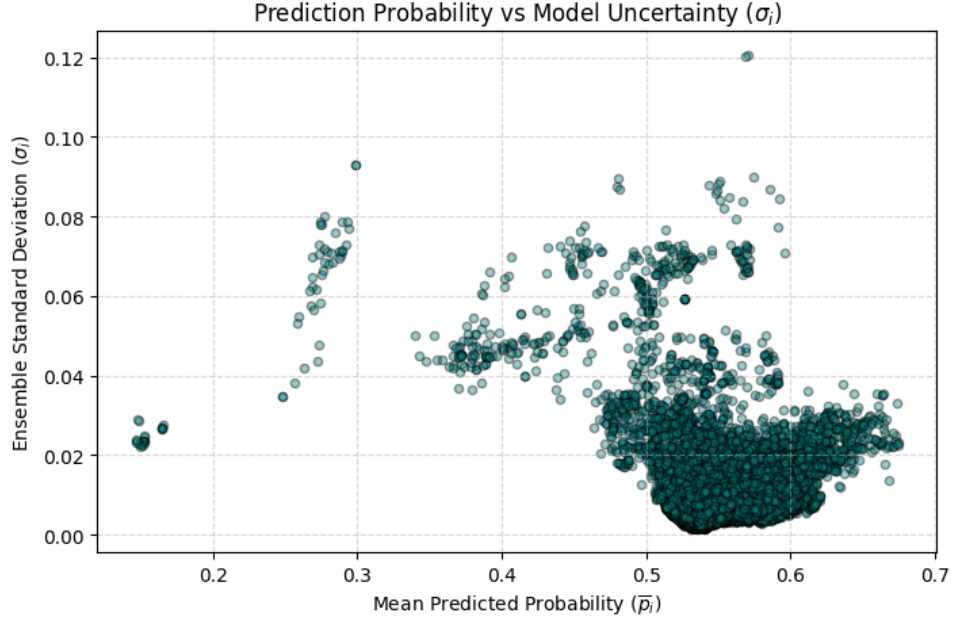


Figure 8: Prediction Probability ($\bar{p}_i$) vs Model Uncertainty (σ_i)

ensemble; rather than sizing these positions equally, a live portfolio should inversely scale capital allocation based on σ_i , actively cutting exposure to highly uncertain predictions to preserve capital.